Exam Practice Platform

Software Engineering Assessment

Full-Stack Developer

Total Duration	Total Marks	Passing Score	Format
14 Hours	100 Points	65 / 100	Take-Home

Candidate Name:	Date:	Submission Link / Repo:

Assessment Overview

This is a take-home assessment designed to evaluate your real-world development skills across the entire stack.

Read through all tasks before starting. Tasks build on each other — completing earlier tasks will make later ones significantly easier.

Layer	Technology
Frontend	Next.js 15 (App Router), TypeScript, Ant Design 5, Zustand, TanStack Query, React Hook Form + Zod, Recharts, SCSS Modules
Backend	Cloudflare Workers + Hono framework (TypeScript)
Database	PostgreSQL via Neon (serverless) — accessed through Drizzle ORM
Cache / Sessions	Cloudflare KV
Object Storage	Cloudflare R2 (pre-signed URLs)
Auth	JWT access tokens (in-memory) + httpOnly refresh cookies + TOTP (2FA)
Monorepo	Turborepo + pnpm workspaces
CI/CD	GitHub Actions — affected-only deploys via Turborepo

General Instructions

Important: Submit a GitHub repository link. All tasks must reside in a monorepo structured with pnpm workspaces and Turborepo. Include a root-level README.md with setup instructions.

What we evaluate:

- Code quality, clarity, and TypeScript strictness
- Correct use of the specified stack — do not substitute libraries unless a task explicitly permits it
- Architecture decisions — structure, naming, separation of concerns
- Security — especially in auth, and data access
- Error handling — edge cases, validation, user-facing error messages
- Testing — coverage and test quality, not just quantity
- Git hygiene — meaningful commits, clear PR descriptions

Time Guidance: Total 14 hours. Tasks are weighted by complexity. Incomplete tasks still earn partial marks — always push what you have. If you run out of time, document what you would have done next.

Phase 1 — Monorepo & Project Scaffolding

Task 1: Monorepo Bootstrap

⏱ 120 min | ⚡ Foundation

Set up the project monorepo from scratch as it would exist in the real codebase. This task tests your ability to configure a professional-grade monorepo before writing any application code.

Scaffold the following workspace structure:

```
assessment/
├── apps/
│   ├── portal/        # Next.js 15 — Student Exam Portal
│   └── admin/         # Next.js 15 + PayloadCMS — Admin Panel
├── services/
│   ├── api-gateway/   # Cloudflare Worker (Hono)
│   ├── auth-svc/      # Cloudflare Worker (Hono)
│   └── papers-svc/    # Cloudflare Worker (Hono)
├── packages/
│   ├── db/            # Drizzle ORM schemas + migrations
│   ├── types/         # Shared TypeScript interfaces/types
│   └── utils/         # Shared utility functions
├── turbo.json
├── pnpm-workspace.yaml
└── package.json
```

#	Requirement	Description
1	**pnpm Workspaces**	Configure pnpm-workspace.yaml to include all apps/, services/, and packages/ directories
2	**Turborepo**	Create turbo.json with a pipeline for build, dev, lint, test, and deploy tasks. The deploy task must depend on build
3	**TypeScript Config**	Create a shared tsconfig.base.json in the root. All packages must extend it. Strict mode must be enabled globally
4	**ESLint + Prettier**	Configure shared ESLint and Prettier configs in packages/. Apply them across all workspaces
5	**Shared Types Pkg**	In packages/types, export at minimum: User, Paper, ExamSession, and PurchaseRecord TypeScript interfaces that will be used across both frontend and backend
6	**Package Naming**	All internal packages must use the @assessment/* naming convention (e.g., @assessment/types, @assessment/db)

Evaluation Area	Points
Correct pnpm workspace configuration	2 pts
Turbo pipeline with correct task dependencies	3 pts
Strict shared TypeScript config extended correctly	2 pts
ESLint + Prettier configured and working	2 pts

Shared types package with correct interfaces	3 pts

Phase 2 — Database Schema & Drizzle ORM

Task 2: Database Schema Design

⏱ 60 min | 🗄 Backend

Design and implement the PostgreSQL schema for the platform using Drizzle ORM inside the packages/db workspace. This is one of the most important tasks — a well-designed schema will make all subsequent tasks significantly easier.

Context: Platform sells MCQ past papers, model papers, Students purchase papers individually or as bundles. When they attempt a paper, an exam session is created with a server-side timer. Results and weak-topic data are stored after submission.

#	Requirement	Description
1	users	id (UUID PK), email (unique), password_hash, full_name, avatar_url, grade_year, preferred_language, two_fa_enabled, two_fa_secret, created_at, updated_at
2	papers	id, title, type (ENUM: past_paper | model_paper), subject, exam_board (ENUM: GCE_A | GCE_O), language, year, price_lkr, is_published, created_at
3	questions	id, paper_id (FK), question_text, options (JSONB — array of {id, text}), correct_option_id, explanation, difficulty (ENUM: easy | medium | hard), topic_tag, order_index
4	purchases	id, user_id (FK), paper_id (FK), amount_paid_lkr, payment_method, payment_ref, purchased_at — enforce UNIQUE(user_id, paper_id)
5	exam_sessions	id, user_id (FK), paper_id (FK), started_at, expires_at, submitted_at (nullable), score_pct (nullable), status (ENUM: in_progress | submitted | expired)
6	session_answers	id, session_id (FK), question_id (FK), selected_option_id (nullable), is_correct (nullable), answered_at — enforce UNIQUE(session_id, question_id)
7	Indexes	Add appropriate indexes on: papers(subject, exam_board, language, type), purchases(user_id), exam_sessions(user_id, status)
8	Migrations	Run drizzle-kit generate to produce a SQL migration file. Commit it to packages/db/migrations/
9	Seed Script	Write a seed.ts that inserts at least: 2 users, 1 paper with 5 questions, and a completed exam session with answers

```
// packages/db/src/schema/papers.ts  — example structure expected
import { pgTable, uuid, text, numeric, boolean, timestamp, pgEnum } from
'drizzle-orm/pg-core';

export const paperTypeEnum = pgEnum('paper_type', ['past_paper', 'model_paper',
'ai_predicted']);
```

```
export const examBoardEnum = pgEnum('exam_board', ['CAIE', 'Edexcel', 'GCE_A', 'GCE_O']);

export const papers = pgTable('papers', {
  id:          uuid('id').defaultRandom().primaryKey(),
  title:       text('title').notNull(),
  type:        paperTypeEnum('type').notNull(),
  // ... complete the rest
});
```

Evaluation Area	Points
All six tables with correct column types and constraints	6 pts
Enums defined and applied correctly	2 pts
Correct FK relationships with cascade rules	2 pts
UNIQUE constraints where specified	1 pt
Indexes on correct columns	2 pts
Migration file generated and valid	2 pts
Working seed script	2 pts

Phase 3 — Backend Services (Cloudflare Workers + Hono)

Task 3: Auth Service — Register & Login

⏱ 120 min | 🔐 Backend

Implement the auth-svc Cloudflare Worker using Hono. This service is responsible for user registration, login, JWT issuance, and token refresh. It will be called by the api-gateway via a Service Binding — never directly by the browser.

Auth Strategy: Access tokens are short-lived JWT (15 min) stored in memory on the client. Refresh tokens are long-lived (7 days), stored in a KV namespace, and set as httpOnly Secure cookies by the api-gateway. On every request, the api-gateway validates the JWT — it never hits the database.

#	Requirement	Description
1	POST /auth/register	Accept { email, password, full_name }. Hash password with bcrypt (cost 12). Insert user. Return 201 with { user_id, email }. Reject duplicate emails with 409.
2	POST /auth/login	Accept { email, password }. Verify bcrypt hash. Issue a signed JWT (HS256) containing { sub: user_id, email, role }. Store refresh token UUID in KV with TTL 7 days. Return { access_token } in body; set refresh_token in httpOnly Secure cookie.
3	POST /auth/refresh	Read refresh token from cookie. Look it up in KV. If valid, issue a new access token. Rotate the refresh token (delete old, insert new KV entry).
4	POST /auth/logout	Delete the refresh token from KV. Clear the cookie.

5	**Zod Validation**	Validate all request bodies with Zod. Return structured 422 errors: { field, message }[] for validation failures.
6	**Hono Middleware**	Create a reusable jwtAuth middleware that validates the Authorization: Bearer <token> header and attaches { user_id, email, role } to the Hono context. Apply it to all protected routes in this service.
7	**Environment Bindings**	Define a typed Env interface for the Worker: { DB: string (Neon URL), KV: KVNamespace, JWT_SECRET: string }. Use it throughout — no any types on env.

```
// services/auth-svc/src/index.ts  — expected entry point structure
import { Hono } from 'hono';
import type { Env } from './types/env';

const app = new Hono<{ Bindings: Env }>();

app.post('/auth/register', async (c) => {
  // your implementation
});

export default app;
```

Evaluation Area	Points
Register endpoint with bcrypt hashing + duplicate rejection	3 pts
Login with correct JWT payload and KV refresh token storage	4 pts
Token refresh with rotation logic	3 pts
Zod validation with structured error response	2 pts
jwtAuth middleware reusable and correctly typed	3 pts

Task 4: Papers Service — Browse, Preview & Filter

⏱ 60 min | 📄 Backend

Implement the papers-svc Worker. This service handles browsing, filtering, and previewing MCQ papers. It is the most read-heavy service in the system — implement it with efficient queries and clear access control.

#	Requirement	Description
1	**GET /papers**	Support query params: subject?, exam_board?, language?, type?, year?. Use Drizzle to build a conditional WHERE clause. Return paginated results (page, limit, total). Default limit: 20.
2	**GET /papers/:id**	Return full paper metadata. If the requesting user has NOT purchased this paper, return questions with is_preview: true on the first 3 questions only. Remaining questions: return { id, order_index, is_preview: false } — no question text or options. If purchased, return all questions in full.

3	POST /papers/:id/issues	Allow authenticated users to report an issue on a paper. Accept { question_id?, description, issue_type: 'wrong_answer' | 'typo' | 'other' }. Store in a paper_issues table (add to schema). Return 201.
4	Full-Text Search	Add GET /papers/search?q= that uses PostgreSQL tsvector full-text search across title, subject, and topic_tag columns. Return ranked results.
5	Access Control	Purchase verification must query the purchases table via Drizzle — never trust client-provided flags. If the user is not authenticated, treat them as unpurchased.

Evaluation Area	Points
Browse with conditional WHERE query correctly built	3 pts
Pagination implemented correctly (offset/limit + total)	2 pts
Preview access control — correct partial data on unpurchased	4 pts
Full-text search with tsvector ranking	3 pts
Issue reporting endpoint with correct validation	2 pts

Task 5: Exam Service — Session Management & Scoring

⏱ 60 min | ⏱ Backend

Implement the exam-svc Worker. This is the most security-sensitive backend service. The exam timer is server-authoritative — stored in Cloudflare KV — and the server scores submissions independently of what the client sends.

Security Requirement: Never trust the client for scoring. When a session is submitted, the server must re-fetch the correct answers from the database and compute the score independently. Client-provided 'scores' or 'correct flags' must be ignored.

#	Requirement	Description
1	POST /exam/sessions	Start a new exam session. Verify the user has purchased the paper. Check no active session exists. Insert exam_sessions row with expires_at = now() + paper duration. Store session expiry in KV: key = session:{id}, value = expires_at ISO string, TTL = paper duration in seconds. Return session_id and expires_at.
2	POST /exam/sessions/:id/answer	Record an answer for a question. Validate: session is active (not expired, not submitted). Validate question belongs to this paper. Upsert into session_answers. Check KV for expiry — if expired, mark session as 'expired' and return 410.
3	POST /exam/sessions/:id/submit	Mark session as submitted. Compute score server-side: fetch all session_answers, join with questions to get correct_option_id, calculate percentage. Update exam_sessions with score_pct and submitted_at. Delete KV session key.
4	GET /exam/sessions/:id/review	Return full post-submission review: each question with selected_option_id, correct_option_id, is_correct, and explanation. Only accessible if session status is 'submitted'. Only accessible by the session owner.

5	GET /exam/sessions/:id/timer	Return { remaining_seconds, expires_at } by reading from KV. Return 410 if session has expired.

Evaluation Area	Points
Session start with purchase verification + KV timer	4 pts
Answer submission with real-time expiry check	3 pts
Server-side scoring independent of client input	4 pts
Post-submission review with correct access control	3 pts

Phase 4 — API Gateway & Cross-Cutting Concerns

Task 6: API Gateway — Routing, Auth & Rate Limiting

⏱ 60 min | 🔀 Infrastructure

Implement the api-gateway Cloudflare Worker. This is the single entry point for all client requests. It handles CORS, JWT validation, rate limiting, and forwards requests to downstream services via Cloudflare Service Bindings.

#	Requirement	Description
1	CORS	Allow origins: localhost:3001 (dev) and app.examapp.com (prod). Read ENVIRONMENT from Worker vars to determine allowed origins dynamically. Always respond to OPTIONS preflight with correct headers.
2	JWT Validation	Validate Authorization: Bearer <token> header for all non-public routes. Public routes (no auth required): POST /auth/register, POST /auth/login, GET /papers, GET /papers/search, GET /papers/:id. Attach decoded user to the forwarded request header as X-User-Id and X-User-Role.
3	Rate Limiting	Use Cloudflare KV to implement sliding window rate limiting: 60 requests per minute per IP for public routes; 120 per minute for authenticated routes. Return 429 with Retry-After header on breach.
4	Service Bindings	Route requests to the correct Worker via env.AUTH_SVC, env.PAPERS_SVC, env.EXAM_SVC. Define a typed Env interface that includes all service bindings as Fetcher types.
5	Structured Errors	Catch all unhandled errors. Return consistent JSON: { error: { code, message, request_id } }. Generate a unique request_id (UUID) for every request and include it in all responses as X-Request-Id header.

Evaluation Area	Points
CORS with environment-aware allowed origins	2 pts
JWT validation applied correctly to protected routes only	3 pts
KV-based sliding window rate limiter	4 pts

Service Bindings with typed Env interface	2 pts
Consistent structured error responses with request ID	2 pts

Phase 5 — Frontend (Next.js 15 + React)

Task 7: Authentication UI & Auth State

⏱ 120 min | 🖥️ Frontend

Implement the authentication screens and global auth state for the Student Exam Portal (apps/portal). This includes the sign-in page, a Zustand auth store, a TanStack Query auth hook, and a protected route wrapper.

#	Requirement	Description
1	**Login Page**	Create /login page in Next.js 15 App Router. Use React Hook Form + Zod for validation. Fields: email, password. Show field-level validation errors inline. On submit, call POST /auth/login. On success, store access_token in Zustand and redirect to /dashboard.
2	**Zustand Auth Store**	Create stores/authStore.ts. State: { user: User | null, accessToken: string | null, isAuthenticated: boolean }. Actions: setAuth(user, token), clearAuth(). The access token must never be stored in localStorage — memory only.
3	**useAuth Hook**	Create hooks/useAuth.ts using TanStack Query. It should: (1) call GET /auth/me on mount if an access token exists in the store, (2) handle 401 responses by calling clearAuth() and redirecting to /login, (3) expose { user, isLoading, isAuthenticated }.
4	**Protected Route**	Create a ProtectedRoute component (or use Next.js middleware) that redirects unauthenticated users to /login. Wrap /dashboard and all child routes with it.
5	**Ant Design + Zod Error Display**	Use Ant Design Form.Item with help and validateStatus to display Zod validation errors. Error messages must appear beneath each field — not as alerts.
6	**Token Refresh Logic**	Add an Axios (or fetch) interceptor that: on a 401 response, attempts POST /auth/refresh, updates the Zustand store with the new token, and retries the original request once. If refresh also fails, call clearAuth() and redirect to /login.

Evaluation Area	Points
Login page with React Hook Form + Zod validation	3 pts
Zustand store — memory-only token, correct types	3 pts
useAuth hook with TanStack Query and 401 handling	3 pts
Protected route wrapper	1 pt
Token refresh interceptor with single retry	3 pts

Task 8: Paper Browse & Exam-Taking UI

⏱ 120 min | 🖥️ Frontend

Build the core student-facing screens: the paper browser, paper detail view, and the live exam-taking screen. These are the highest-traffic screens in the product.

#	Requirement	Description
1	**Paper Browser (/papers)**	Use TanStack Query to fetch GET /papers with subject, exam_board, and type filters. Implement filter UI using Ant Design Select components. Display results in a responsive Ant Design Card grid. Show a skeleton loader while fetching. Implement pagination using Ant Design Pagination — sync with URL query params.
2	**Paper Detail (/papers/:id)**	Fetch paper metadata and questions. Show full questions for purchased papers; show a blur/lock overlay for locked questions with a 'Purchase to Unlock' CTA. Display a difficulty badge (Easy / Medium / Hard) on each question card.
3	**Exam Screen (/exam/:sessionId)**	Build the live exam UI. Show a question navigator sidebar (question number grid — answered/unanswered state). Show one question at a time with 4 radio options. Show a live countdown timer that reads remaining_seconds from GET /exam/sessions/:id/timer. When timer reaches 0, disable all inputs and auto-submit.
4	**Results Screen (/exam/:sessionId/results)**	After submission, show: total score as a circular progress chart (using Recharts RadialBarChart), a topic-wise breakdown bar chart, and a question-by-question review list (question text, selected answer, correct answer, explanation). Correct answers highlighted green; wrong answers highlighted red.
5	**Optimistic Updates**	When a user selects an answer option, immediately update the question navigator sidebar state (mark as answered) without waiting for the API response. Revert on API error.

Evaluation Area	Points
Paper browser with filters, cards, and pagination synced to URL	4 pts
Preview/lock access control correctly reflected in UI	2 pts
Live exam screen with question navigator and countdown timer	4 pts
Results with Recharts score + topic breakdown charts	3 pts
Optimistic update on answer selection with revert on error	2 pts

Phase 6 — Testing

Task 9: Unit & Integration Tests

⏱ 60 min | 🧪 Testing

Write tests that give a hiring team confidence that the critical paths of your code are correct and will remain correct during future changes. Focus on quality and edge cases over quantity.

Stack: Use Vitest for all tests. Use @cloudflare/workers-types and miniflare for Worker tests. Use React Testing Library for component tests.

#	Requirement	Description
1	**Auth Service — Unit Tests**	Test POST /auth/register: (1) valid registration returns 201 with user_id; (2) duplicate email returns 409; (3) missing email field returns 422 with field error. Mock the database — do not make real DB calls in unit tests.
2	**Exam Scoring — Unit Test**	Extract the score computation logic into a pure function computeScore(answers: SessionAnswer[], questions: Question[]): number. Write at least 5 unit tests: all correct, all wrong, partial, empty answers array, and question not answered (null selected_option_id treated as wrong).
3	**Auth Store — Unit Test**	Test the Zustand auth store: setAuth persists correct state; clearAuth resets to initial state; isAuthenticated is true after setAuth and false after clearAuth.
4	**Login Component — Integration Test**	Using React Testing Library: render the Login page, fill in email and password using fireEvent, submit the form, assert the API call was made with correct payload (mock the API), and assert redirect to /dashboard on success.
5	**Rate Limiter — Unit Test**	Test the sliding window rate limiter logic in isolation: (1) first 60 requests pass; (2) 61st request returns 429; (3) after 1 minute window elapses, counter resets. Mock KV.

Evaluation Area	Points
Auth service unit tests with DB mock	3 pts
computeScore pure function + 5 edge case tests	4 pts
Zustand store unit tests	2 pts
Login component integration test with API mock	3 pts
Rate limiter unit test with KV mock	3 pts

Phase 7 — DevOps, CI/CD & Architecture

Task 10: CI/CD Pipeline & Architecture Review

⏱ 60 min | ⚙ DevOps

The final phase tests your understanding of the deployment pipeline and your ability to reason about architecture. Complete both parts — the GitHub Actions workflow and the written architecture review.

Part A — GitHub Actions Workflow (15 pts)

Create .github/workflows/deploy.yml. The pipeline must:

- Trigger on push to main
- Set up Node 20, pnpm 9, and restore the pnpm store cache

- Run pnpm install --frozen-lockfile
- Run pnpm turbo build --filter='[HEAD^1]' to build only changed packages
- Run pnpm turbo test --filter='[HEAD^1]' and fail the build if any test fails
- Run pnpm turbo deploy --filter='[HEAD^1]' --filter='./services/*' to deploy only changed Workers, using CLOUDFLARE_API_TOKEN from GitHub secrets
- Skip the deploy step for pull requests (deploy on push to main only)
- Cache the Turborepo build artifacts using actions/cache with a cache key derived from the git SHA

```
# .github/workflows/deploy.yml
# Implement the full workflow based on the requirements above.
# The pipeline must build and test affected packages, then deploy only
# affected Cloudflare Workers on push to main.
```

Part B — Architecture Review (10 pts)

Create ARCHITECTURE_REVIEW.md in the repo root. Answer all three questions with clear, technical reasoning. Target 150-250 words per answer.

Question 1 — Scaling the Exam Timer: The server-authoritative exam timer is stored in Cloudflare KV. KV has eventual consistency — writes may take up to 60 seconds to propagate globally. Describe how this eventual consistency could cause a problem during an exam session, give a concrete failure scenario, and explain how you would mitigate it.

Question 2 — Service Binding vs HTTP: Platform uses Cloudflare Service Bindings for inter-service communication instead of HTTP requests. Explain two concrete advantages and one concrete limitation of this approach compared to making standard HTTP requests between services.

Evaluation Area	Points
GitHub Actions workflow — correct trigger, pnpm + Node setup, cache	3 pts
Turborepo affected-only build and test steps	4 pts
Correct conditional deploy with secrets, skip on PR	4 pts
Architecture Q1 — KV consistency risk + mitigation	5 pts
Architecture Q2 — Service Bindings trade-offs	5 pts

Scoring Summary

Phase	Task	Area Tested	Max Pts	Score
1	Task 1 — Monorepo Bootstrap	Tooling & Config	12	___
2	Task 2 — Database Schema & Drizzle	DB Design	17	___
3	Task 3 — Auth Service	Backend Auth	15	___
3	Task 4 — Papers Service	Backend API	14	___
3	Task 5 — Exam Service	Backend Logic	14	___
4	Task 6 — API Gateway	Infrastructure	13	___
5	Task 7 — Auth UI & State	Frontend	13	___
5	Task 8 — Browse & Exam UI	Frontend	15	___
6	Task 9 — Unit & Integration Tests	Testing	15	___
7	Task 10 — CI/CD & Architecture	DevOps + Design	21	___
	TOTAL		149*	___

*Total available points: 149. Final score is scaled to 100 (divide by 1.49). Passing threshold: 65/100.

90–100 | Exceptional	75–89 | Strong	65–74 | Good	< 65 | Average